



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

**THIẾT BỊ ĐEO THEO DÕI SỨC KHỎE, PHÁT HIỆN TẾ
NGÃ VÀ ĐỊNH VỊ DÙNG ESP32-S3**

BÁO CÁO CHUNG TỔNG KẾT NHÓM

THÀNH VIÊN THỰC HIỆN

STT	Họ và tên	MSSV
1	Phạm Hùng Tiến	24207043
2	Hồ Sĩ Phú	24207101
3	Quách Gia Thịnh	24207105

Mã nguồn đồ án: github.com/PhamHungTien/esp32-s3-health-monitor

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

MỤC LỤC

1	Tóm tắt đồ án	3
2	Đối chiếu yêu cầu nội bài	3
3	Yêu cầu và phạm vi	4
4	Kiến trúc hệ thống	4
4.1	Bảng kết nối	5
4.2	Hình ảnh các linh kiện chính	5
4.3	Nguyên mẫu hoạt động trên mạch thử	6
5	Thiết kế phần mềm	6
5.1	Tổ chức chương trình	6
5.2	Luồng vòng lặp chính	8
5.3	Giao diện OLED	8
5.4	Giao diện web qua Wi-Fi	8
6	Thuật toán các mô-đun	8
6.1	Nhịp tim và SpO ₂	9
6.2	Phát hiện té ngã	9
6.3	Đếm bước	9
6.4	GPS	10
7	Các đoạn mã quyết định	10
7.1	Phân biệt FIFO rỗng với lỗi MAX30102	10
7.2	Xác nhận BPM trước khi hiển thị	10
7.3	Điều kiện bước chân và té ngã	11
7.4	Loại câu NMEA sai checksum	13
7.5	Đổi và kiểm tra tọa độ	13
7.6	Tách GGA và RMC	14
7.7	Thu UART và phân loại trạng thái GPS	15
8	Phân công thực hiện	16
9	Kiểm thử và kết quả	16
10	Đánh giá rủi ro và giới hạn	17
11	Hướng phát triển	18
12	Kết luận	18

Tài liệu tham khảo

18

1. Tóm tắt đồ án

Nhóm xây dựng thiết bị nguyên mẫu dùng ESP32-S3 để đo nhịp tim, ước lượng nồng độ oxy trong máu, đếm bước, hiển thị trên OLED, phát hiện té ngã, lấy vị trí GPS và cung cấp dashboard qua Wi-Fi cục bộ. Chương trình chỉ dùng thành phần thuộc ESP32 Arduino Core; driver cảm biến, xử lý tín hiệu, parser và giao diện web được viết trong mã nguồn của nhóm.

Kết quả chính

Ba thiết bị I2C được nhận đồng thời; OLED và dashboard hiển thị dữ liệu sức khỏe; MAX30102 phát hiện ngón tay và BPM hội tụ; IMU trả gia tốc nghỉ xấp xỉ 1g; GPS bắt FIX với 4 vệ tinh ở lần chạy cuối. ESP32-S3 phát mạng health-monitor, phục vụ giao diện tại 192.168.4.1 và API JSON cập nhật mỗi 0,25 giây. Kênh LEDC cho buzzer và hai thao tác web đặt lại bước/hủy cảnh báo đã được cài đặt. Đếm bước, SpO₂ và té ngã vẫn cần thêm hiệu chuẩn hoặc phép thử thực tế.

2. Đối chiếu yêu cầu nội bài

Phần cuối buổi học ngày 23/07 nêu ba loại hồ sơ và yêu cầu phần việc cá nhân không trùng nhau. Nhóm chuẩn bị các tệp theo bảng sau.

Hồ sơ/yêu cầu	Cách thực hiện	Đối chiếu
Bảng phân công và đánh giá	Một tệp chung, nêu tính năng, kết quả và tỷ lệ đóng góp của từng người	Đạt
Báo cáo công việc cá nhân	Ba tệp riêng; mỗi tệp mô tả phần mã, phép thử và hạn chế của đúng mô-đun được giao	Đạt
Báo cáo chung tổng kết	Một tệp dùng chung, tổng hợp kiến trúc, kết quả và phần chưa hoàn thiện	Đạt
Không tính “tìm tài liệu” là nhiệm vụ	Mỗi thành viên chịu một khối kỹ thuật có đầu ra chạy được; tài liệu chỉ nằm ở mục tham khảo	Đạt
Không trùng phần việc cá nhân	Tích hợp/hiển thị; PPG; IMU–GPS được tách thành ba phạm vi khác nhau	Đạt
Báo cáo đúng tiến độ thực tế	SpO ₂ , đếm bước và té ngã được ghi rõ phần còn phải hiệu chuẩn/thử thêm	Đạt

3. Yêu cầu và phạm vi

Yêu cầu	Cách đáp ứng	Trạng thái
Đo nhịp tim	Thu PPG IR 100 Hz, dò nhịp, làm mượt BPM	Đạt
Ước lượng SpO ₂	Ratio-of-ratios từ RED/IR, cờ chất lượng	Cần hiệu chuẩn
Phản hồi nhanh	BPM sau RR hợp lệ đầu tiên; SpO ₂ dùng cửa sổ 64 mẫu	Đạt
Đếm bước	Lọc gia tốc động, kiểm tra chu kỳ bước	Đã cài đặt
Hiển thị tại chỗ	Driver SSD1306, BPM/SpO ₂ cỡ lớn	Đạt
Hiển thị trên web	Wi-Fi AP, captive portal, API JSON và dashboard responsive	Đã cài đặt
Cảnh báo té ngã	Chuỗi rơi tự do–và đập–bất động	Đã cài đặt
Định vị	UART, checksum NMEA, parser GGA/RMC	Đạt
Âm báo	PWM LEDC trên GPIO42, hai âm xen kẽ khi ALERT	Đã cài đặt
Không dùng thư viện ngoài	Driver và parser viết trong sketch	Đạt

4. Kiến trúc hệ thống

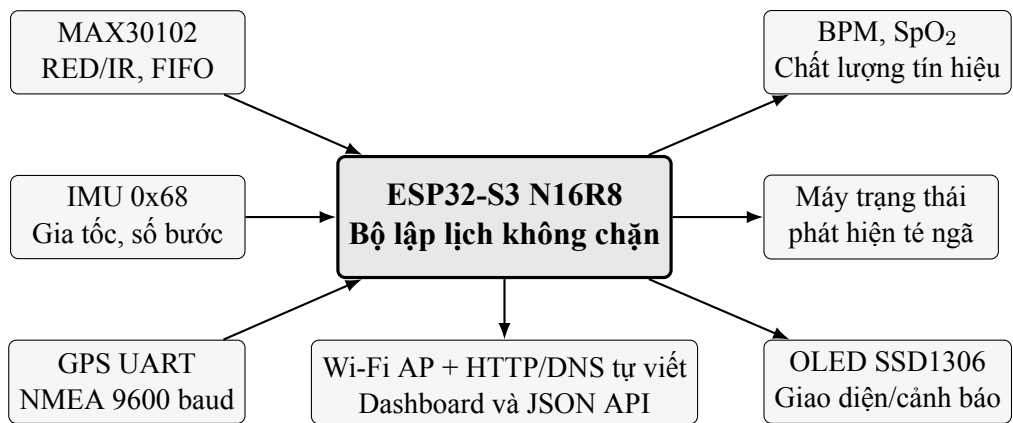


Figure 1: Sơ đồ khối kiến trúc tổng thể của thiết bị.

4.1. Bảng kết nối

Khối	Giao tiếp	Tài nguyên	Ghi chú
OLED SSD1306	I2C, 0x3C	SDA GPIO8, SCL GPIO9	VCC 3,3 V; GND chung
MAX30102	I2C, 0x57	Chung SDA/SCL	VCC 3,3 V; FIFO 100 Hz
IMU tương thích MPU	I2C, 0x68	Chung SDA/SCL	WHO_AM_I 0x74; đọc gia tốc từ 0x3B
GPS	UART1 phần cứng	RX GPIO21, TX GPIO47	GPS TX nối GPIO21; GPS RX nối GPIO47; 9600 baud
Loa piezo/buzzer	PWM LEDC	SPK+ GPIO42, SPK- GND	Cảnh báo té ngã; camera không dùng
USB	USB native	USB- Serial/JTAG	Nạp và nhật ký
Dashboard web	Wi-Fi 2,4 GHz	AP 192.168.4.1	SSID health-monitor; mật khẩu 99999999

Ràng buộc bo camera

GPIO8 và GPIO9 cũng thuộc nhóm tín hiệu camera trên bo Freenove ESP32-S3 Camera. Trong cấu hình đồ án, camera không được sử dụng và phải tách khỏi hai chân này để tránh tranh chấp với bus I2C.

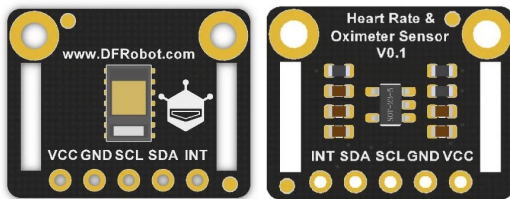
4.2. Hình ảnh các linh kiện chính



(a) Bo điều khiển Freenove ESP32-S3 N16R8.



(b) OLED SSD1306 128x64 tham chiếu.



(c) Bo breakout MAX30102 tham chiếu.



(d) Mô-đun GPS UART tham chiếu.

Figure 2: Nhóm linh kiện chính của nguyên mẫu. Ảnh bo điều khiển là đúng dòng sản phẩm; ảnh OLED, MAX30102 và GPS thể hiện loại linh kiện/giao tiếp, không dùng để khẳng định nhà sản xuất của các bo breakout đang lắp. Nguồn: Freenove, Waveshare và DFRobot.

4.3. Nguyên mẫu hoạt động trên mạch thử

Hình 3 xác nhận chuỗi phần cứng đã được lắp và chạy đồng thời trên một nguyên mẫu thống nhất, thay vì chỉ kiểm tra từng mô-đun riêng lẻ. Đây là cấu hình được sử dụng để kiểm tra hiển thị OLED, tín hiệu PPG, dữ liệu chuyển động, luồng GPS và cảnh báo âm thanh.

5. Thiết kế phần mềm

5.1. Tổ chức chương trình

Theo yêu cầu của đồ án, chương trình chỉ dùng các thành phần có sẵn trong ESP32 Arduino Core như Wire, UART phần cứng và `millis()`. Các phần được cài đặt trong sketch gồm:

- chuỗi lệnh khởi tạo SSD1306, framebuffer, font 5x7 và đồ họa;
- cấu hình thanh ghi, đọc FIFO và xử lý tín hiệu MAX30102;
- đọc thanh ghi IMU và máy trạng thái phát hiện té ngã;
- lọc gia tốc và xác định chuỗi bước đi;
- bộ đệm dòng, checksum, tách trường và đổi tọa độ NMEA;
- Wi-Fi Access Point, phản hồi DNS captive portal, bộ phân tích/định tuyến HTTP, API JSON và dashboard responsive do nhóm tự viết;

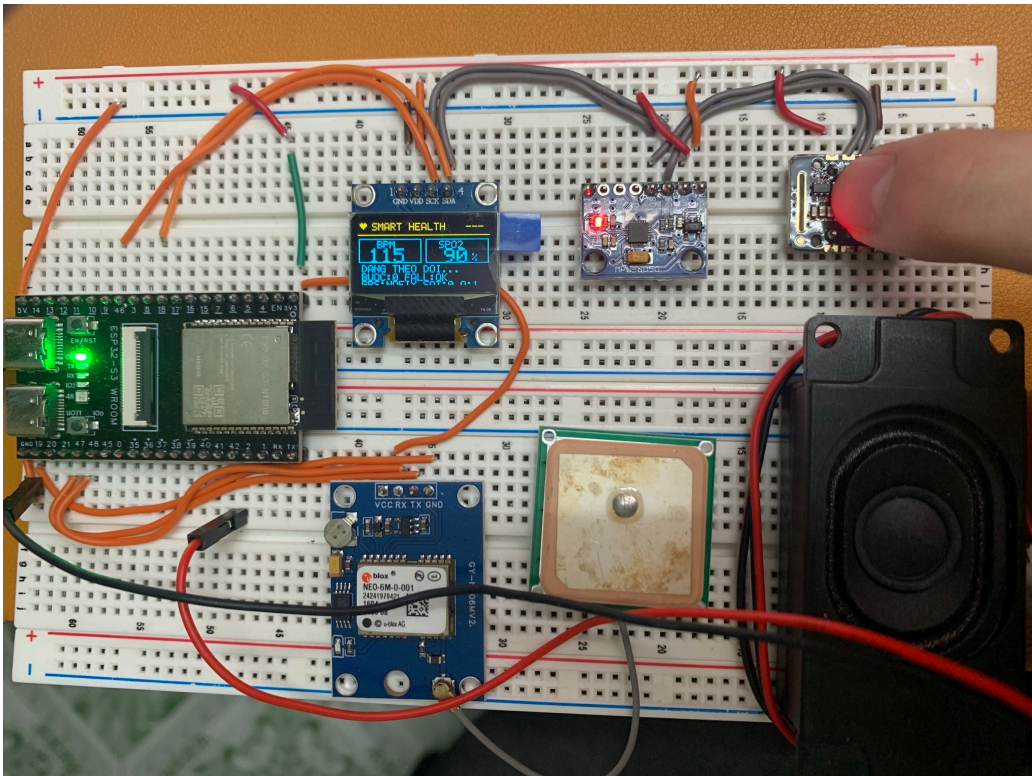


Figure 3: Nguyên mẫu thực tế của nhóm lắp trên breadboard gồm ESP32-S3, OLED, MAX30102, IMU, GPS và buzzer. OLED đang nhận dữ liệu từ firmware tại thời điểm chụp; các số hiển thị trong ảnh chỉ minh họa trạng thái hoạt động, không được dùng làm số liệu hiệu chuẩn y tế. Ảnh do nhóm chụp.

- bộ lập lịch theo thời gian và cơ chế phát hiện lại thiết bị.

Tệp tiêu đề	Nguồn	Mục đích
Arduino.h	ESP32 Arduino Core	GPIO, thời gian, LEDC và hàm nền tảng
Wire.h	ESP32 Arduino Core	Chỉ thực hiện giao tiếp I2C được giảng viên cho phép
HardwareSerial.h	ESP32 Arduino Core	Nhận byte UART từ GPS
WiFi.h	ESP32 Arduino Core	Điều khiển radio, Access Point và socket TCP
WiFiUdp.h	ESP32 Arduino Core	Truyền/nhận datagram UDP mức thấp
math.h	Thư viện chuẩn C/C++	Căn bậc hai, trị tuyệt đối

Sketch không dùng Adafruit SSD1306/GFX, SparkFun MAX3010x, TinyGPS++, thư viện MPU, WebServer, DNSServer hay thư viện xử lý tín hiệu. Driver cảm biến, thuật toán, parser NMEA, parser/router HTTP, JSON và phản hồi DNS đều nằm trong mã nguồn của nhóm. `WiFi.h`

và `WiFiUdp.h` chỉ là lớp phần cứng/socket bắt buộc thuộc ESP32 Arduino Core.

5.2. Luồng vòng lặp chính

Ở mỗi vòng lặp, chương trình xử lý nhanh DNS/HTTP đang chờ, đọc hết byte GPS và rút FIFO PPG. Theo các mốc thời gian riêng, chương trình cập nhật IMU, máy trạng thái té ngã, OLED và nhật ký. Không có trì hoãn dài nên các nguồn dữ liệu tốc độ khác nhau và dashboard vẫn hoạt động đồng thời.

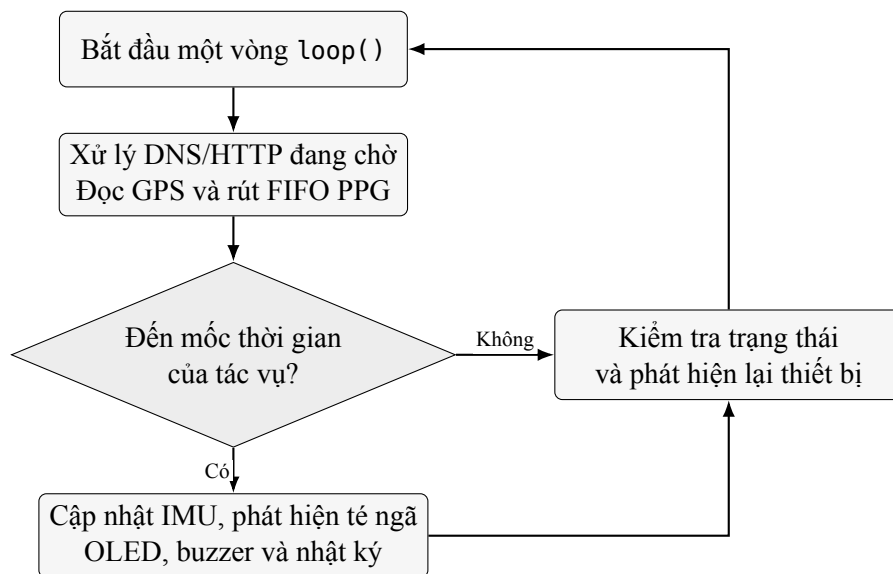


Figure 4: Sơ đồ khối luồng xử lý không chặn trong vòng lặp chính.

5.3. Giao diện OLED

Màn hình chính ưu tiên BPM và SpO_2 bằng ký tự cỡ lớn. Các hàng dưới hiển thị hướng dẫn đo, số bước, trạng thái té ngã, GPS, số vệ tinh/gia tốc hoặc tọa độ luân phiên. Khi có té ngã, toàn màn hình chuyển sang cảnh báo khẩn; nếu GPS vừa mất fix, màn hình dùng tọa độ hợp lệ gần nhất và ký hiệu L-LAT/L-LON. Các số chân đầu nối không xuất hiện trên giao diện người dùng.

5.4. Giao diện web qua Wi-Fi

ESP32-S3 tạo Access Point health-monitor với mật khẩu 99999999. Sau khi kết nối, người dùng mở `http://192.168.4.1`. Tài nguyên HTML/CSS/JavaScript nằm trong flash và không dùng CDN. API tại `/api/status` trả BPM, SpO_2 , chất lượng PPG, dữ liệu thô, số bước, trạng thái té ngã, gia tốc, trạng thái từng mô-đun, GPS, vệ tinh, tọa độ, uptime và số máy khách.

Hai endpoint POST cho phép đặt lại số bước và hủy cảnh báo. Dashboard chỉ hiện số sức khỏe khi cờ hợp lệ tương ứng là đúng; liên kết bản đồ chỉ được mở khi hệ thống đã từng nhận tọa độ hợp lệ.

6. Thuật toán các mô-đun

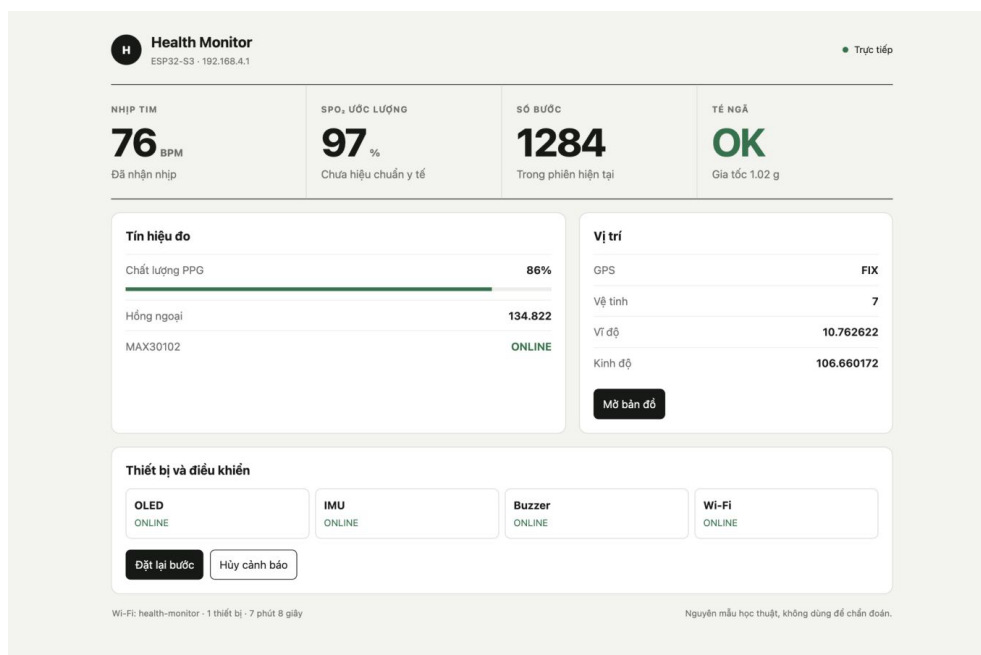


Figure 5: Dashboard web tinh gọn được render với dữ liệu minh họa để kiểm tra bố cục. Khi chạy trên ESP32-S3, trang lấy dữ liệu thực từ API JSON mỗi 0,25 giây.

6.1. Nhịp tim và SpO₂

MAX30102 cung cấp hai kênh quang RED/IR ở 100 Hz. Trong 0,35 giây đầu, bộ lọc bám nhanh theo mức DC để xung đặt tay không làm ngưỡng nhịp tăng sai. Sau đó, hai cạnh trái dấu tạo BPM sơ bộ từ nửa chu kỳ; cạnh cùng cực tính kế tiếp xác nhận RR đầy đủ và thay số sơ bộ. Nếu không có RR xác nhận trong 1,8 giây, số sơ bộ tự bị hủy. SpO₂ dùng tỷ lệ giữa AC/DC trong cửa sổ 64 mẫu, được xét độc lập với BPM và có thể xuất sau khoảng một giây nếu tín hiệu quang đạt điều kiện.

Ngưỡng nhận tay là IR lớn hơn 10.000; sau khi đã nhận, ngưỡng nhả giảm còn 7.000 để tạo vùng trễ. Hụt tín hiệu dưới 450 ms chưa xóa số đo, nhưng nếu tay quay lại sau ít nhất 180 ms thì cạnh, cực tính và RR cũ bị xóa để bắt đầu phiên mới. Khi đặt lại tay hoặc bộ dò chưa tìm được BPM sau 3,5 giây, vòng lặp yêu cầu khởi tạo lại MAX30102 và FIFO; vì vậy việc bắt lại không phụ thuộc thao tác mở Serial Monitor.

6.2. Phát hiện té ngã

Độ lớn gia tốc ba trục được đưa qua chuỗi trạng thái NORMAL, FREEFALL, IMPACT và ALERT. Cảnh báo chỉ sinh ra nếu ba hiện tượng rơi tự do, va đập và ít chuyển động xuất hiện đúng thứ tự trong các cửa sổ thời gian cho phép.

6.3. Đếm bước

Thành phần gia tốc chậm được dùng làm đường nền. Chương trình đếm các đỉnh chuyển động có khoảng cách 280–1.800 ms và chỉ bắt đầu cộng khi có hai đỉnh liên tiếp. Số bước được giữ trong RAM cho đến khi thiết bị khởi động lại.

6.4. GPS

GPS dùng UART1 ở 9.600 baud và được đọc hết byte đang chờ trong mỗi vòng lặp. Bộ đệm tự gom câu từ ký tự “\$” đến cuối dòng, kiểm tra checksum XOR rồi mới tách tối đa 16 trường. GGA cung cấp chất lượng fix, số vệ tinh và tọa độ; RMC chỉ được nhận khi trạng thái bằng A. Tọa độ ddmm.mmmm/ddmm.mmmm được đổi sang độ thập phân, kiểm tra bán cầu và giới hạn vĩ/kinh độ trước khi lưu. Hệ thống tách WAIT, LOST, NOFIX và FIX theo bộ đếm byte cùng thời điểm nhận dữ liệu/fix gần nhất để tránh hiểu nhầm chưa bắt vệ tinh là lỗi dây.

7. Các đoạn mã quyết định

Các đoạn dưới đây được trích từ sketch nộp kèm. Chúng thể hiện những điều kiện trực tiếp quyết định đầu ra của hệ thống; mã đầy đủ còn chứa phần khởi tạo, vẽ giao diện và nhật ký kiểm tra.

7.1. Phân biệt FIFO rỗng với lỗi MAX30102

```

1 // Đọc một mẫu từ FIFO MAX30102 và lưu lại kết quả trả về.
2 int8_t result = MAX30102_ReadFIFO();
3
4 // Thoát vòng đọc khi FIFO hiện không có mẫu mới; đây không phải lỗi cảm biến.
5 if (result == FIFO_NO_DATA) break;
6
7 // Đi vào nhánh xử lý riêng khi giao dịch I2C với MAX30102 thất bại.
8 if (result == FIFO_BUS_ERROR) {
9
10 // Đánh dấu cảm biến quang đang mất kết nối để cơ chế phục hồi có thể khởi tạo lại.
11 maxOK = false;
12
13 // Xóa BPM, SpO2 và trạng thái bộ lọc để không giữ số đo của phiên cũ.
14 PPG_ResetMetrics();
15
16 // Đặt cả hai mẫu quang thô về 0, tránh giao diện hiểu dữ liệu cũ là mẫu mới.
17 ppg.redRaw = ppg.irRaw = 0;
18
19 // Dừng vòng đọc FIFO ngay sau lỗi bus.
20 break;
21
22 // Kết thúc nhánh xử lý lỗi giao tiếp.
23 }
```

Đoạn mã 1: Không dùng dữ liệu PPG cũ khi cảm biến mất kết nối

7.2. Xác nhận BPM trước khi hiển thị

```

1 // Dừng ngay RR đầy đủ nếu giá trị trước là sơ bộ, chưa hợp lệ hoặc bộ dò vừa bắt đầu lại.
2 if (ppg.heartRateProvisional || !ppg.heartRateValid ||
3     state.replaceBpmOnNextInterval) {
4     ppg.bpm = instantBPM;
5     ppg.heartRateValid = true;
6     ppg.heartRateProvisional = false;
7     state.replaceBpmOnNextInterval = false;
8
9     // Chỉ làm mượt nhịp kế tiếp khi lệch không quá 18 BPM so với số đang hiển thị.
10 } else if (fabsf(instantBPM - ppg.bpm) <= 18.0f) {
11
12     // Giữ 90% giá trị cũ và nhận 10% giá trị mới để OLED ổn định hơn khi ngón tay rung nhẹ.
13     ppg.bpm = 0.90f * ppg.bpm + 0.10f * instantBPM;
14 }

```

Đoạn mã 2: Khoảng RR hợp lệ đầu tiên tạo BPM ban đầu

7.3. Điều kiện bước chân và té ngã

```

1 // Chỉ xét một bước khi đã thấy đáy âm và tín hiệu động vượt ngưỡng dương 0,11 g.
2 if (peakArmed && dynamicFiltered > 0.11f &&
3
4     // Yêu cầu cách đỉnh trước hơn 280 ms và tổng gia tốc nhỏ hơn 1,8 g để loại rung hoặc va đập.
5     now - lastPeakMs > 280 && totalG < 1.8f) {
6
7     // Tính thời gian giữa hai đỉnh; khi chưa có đỉnh trước thì trả về 0.
8     uint32_t interval = lastPeakMs == 0 ? 0 : now - lastPeakMs;
9
10    // Xác nhận chu kỳ nằm trong miền 280–1.800 ms của một nhịp bước hợp lý.
11    if (interval >= 280 && interval <= 1800) {
12
13        // Kiểm tra chuỗi đi bộ đã được xác nhận hay mới bắt đầu.
14        if (walkingSequence) {
15
16            // Nếu chuỗi đang hợp lệ, cộng thêm đúng một bước cho đỉnh mới.
17            stepCount++;
18
19            // Đi vào nhánh khởi tạo khi vừa có cặp đỉnh hợp lệ đầu tiên.
20        } else {
21
22            // Cộng hai bước để tính cả đỉnh đầu tiên từng được giữ lại để xác nhận chuỗi.
23            stepCount += 2;
24
25            // Đánh dấu các đỉnh tiếp theo thuộc cùng một chuỗi đi bộ.
26            walkingSequence = true;
27
28            // Kết thúc lựa chọn cách cập nhật bộ đếm bước.
29        }
30
31        // Kết thúc điều kiện chu kỳ bước hợp lệ.
32    }
33
34    // Kết thúc điều kiện xác nhận đỉnh dương.
35 }
36
37 // Máy trạng thái dưới đây giữ đủ chuỗi NORMAL–FREE_FALL–IMPACT–ALERT như firmware.
38 switch (fallState) {
39     case FALL_NORMAL:
40         if (accelerationG < 0.45f) {
41             if (lowGStartMs == 0) lowGStartMs = now;
42             if (now - lowGStartMs >= 120) {
43                 fallState = FALL_FREE_FALL;
44                 fallStateMs = now;

```

```
33     }
34   } else {
35     lowGStartMs = 0;
36   }
37   // Va đập trực tiếp trên 2,8 g cũng có thể mở ngay giai đoạn kiểm tra bất động.
38   if (accelerationG > 2.8f) {
39     fallState = FALL_IMPACT;
40     fallStateMs = now;
41   }
42   break;
43
44   case FALL_FREE_FALL:
45     // Sau rơi tự do phải có va đập trên 2,2 g trong 1.200 ms; quá hạn thì hủy chuỗi.
46     if (accelerationG > 2.2f) {
47       fallState = FALL_IMPACT;
48       fallStateMs = now;
49     } else if (now - fallStateMs > 1200) {
50       fallState = FALL_NORMAL;
51       lowGStartMs = 0;
52     }
53     break;
54
55   case FALL_IMPACT:
56     // Sau va đập, chỉ cảnh báo khi bất động hơn 1.800 ms và tổng gia tốc trở về gần 1 g.
57     if (now - fallStateMs > 1800 && motionLevel < 0.08f &&
58         accelerationG > 0.72f && accelerationG < 1.28f) {
59       fallState = FALL_ALERT;
60       fallStateMs = now;
61       Serial.println(
62         "[ALERT] PHAT HIEN TE NGA - CAN KIEM TRA NGUOI DUNG");
63     } else if (now - fallStateMs > 5000) {
64       fallState = FALL_NORMAL;
65     }
66     break;
67
68   case FALL_ALERT:
69     // Giữ cảnh báo để OLED, buzzer và dashboard tiếp tục phản ánh cùng một trạng thái.
70     break;
71 }
```

Đoạn mã 3: Ngưỡng động học dùng cho hai chức năng IMU

7.4. Loại câu NMEA sai checksum

```

1 // Đổi một chữ số hexadecimal sang giá trị 0-15; ký tự lạ trả -1.
2 int8_t NMEA_HexNibble(char c) {
3     if (c >= '0' && c <= '9') return c - '0';
4     if (c >= 'A' && c <= 'F') return c - 'A' + 10;
5     if (c >= 'a' && c <= 'f') return c - 'a' + 10;
6     return -1;
7 }
8
9 // Chỉ nhận câu bắt đầu bằng đô-la, có dấu sao và đủ hai chữ số checksum.
10 bool NMEA_ChecksumOK(const char *sentence) {
11     if (!sentence || sentence[0] != '$') return false;
12     const char *star = strchr(sentence, '*');
13     if (!star || strlen(star) < 3) return false;
14     int8_t high = NMEA_HexNibble(star[1]);
15     int8_t low = NMEA_HexNibble(star[2]);
16     if (high < 0 || low < 0) return false;
17
18     // XOR toàn bộ phần thân giữa ký tự đô-la và dấu sao.
19     uint8_t checksum = 0;
20     for (const char *p = sentence + 1; p < star; p++) {
21         checksum ^= (uint8_t)*p;
22     }
23
24     // Ghép hai nửa byte và so sánh với checksum tự tính.
25     uint8_t expected = (uint8_t)((high << 4) | low);
26     return checksum == expected;
27 }

```

Đoạn mã 4: Hàm kiểm tra đầy đủ checksum NMEA

7.5. Đổi và kiểm tra tọa độ

```

1 // Tách phần độ và phút, sau đó đổi sang độ thập phân.
2 double NMEA_ToDegrees(const char *value, char hemisphere) {
3     if (!value || !*value) return 0.0;
4     double raw = atof(value);
5     int degrees = (int)(raw / 100.0);
6     double minutes = raw - degrees * 100.0;
7     double result = degrees + minutes / 60.0;
8
9     // Bán cầu Nam hoặc Tây làm kết quả mang dấu âm.
10    if (hemisphere == 'S' || hemisphere == 'W') result = -result;
11    return result;
12 }
13
14 // Loại trường rỗng, ký hiệu bán cầu sai và tọa độ vượt miền địa lý.
15 bool GPS_StorePosition(const char *latitude, char northSouth,
16                        const char *longitude, char eastWest) {
17     if (!latitude || !longitude || !latitude[0] || !longitude[0]) return false;
18     if (northSouth != 'N' && northSouth != 'S') return false;
19     if (eastWest != 'E' && eastWest != 'W') return false;
20     double parsedLatitude = NMEA_ToDegrees(latitude, northSouth);
21     double parsedLongitude = NMEA_ToDegrees(longitude, eastWest);
22     if (fabs(parsedLatitude) > 90.0 ||

```

```

22         fabs(parsedLongitude) > 180.0) return false;
23 // Lưu tọa độ, hai cờ vị trí và thời điểm fix mới nhất cho mọi giao diện.
24     gpsLatitude = parsedLatitude;
25     gpsLongitude = parsedLongitude;
26     gpsFix = true;
27     gpsPositionKnown = true;
28     lastGPSTimeMs = millis();
29     return true;
30 }

```

Đoạn mã 5: Đổi tọa độ NMEA và chỉ lưu giá trị hợp lệ

7.6. Tách GGA và RMC

```

1 // Không xử lý tiếp nếu câu NMEA sai checksum.
2 void GPS_ParseSentence(const char *sentence) {
3     if (!NMEA_ChecksumOK(sentence)) return;
4     char copy[96];
5     strncpy(copy, sentence, sizeof(copy));
6     copy[sizeof(copy) - 1] = '\0';
7     char *star = strchr(copy, '*');
8     if (star) *star = '\0';
9
10 // Tự tách tối đa 16 trường bằng dấu phẩy, không dùng thư viện parser GPS.
11     char *fields[16] = {nullptr};
12     uint8_t count = 1;
13     fields[0] = copy;
14     for (char *p = copy; *p && count < 16; p++) {
15         if (*p == ',') {
16             *p = '\0';
17             fields[count++] = p + 1;
18         }
19     }
20     if (count == 0) return;
21     bool isGGA = strstr(fields[0], "GGA") != nullptr;
22     bool isRMC = strstr(fields[0], "RMC") != nullptr;
23
24 // GGA cập nhật số vệ tinh; quality bằng 0 hoặc tọa độ sai làm mất fix.
25     if (isGGA && count > 7) {
26         uint8_t quality = (uint8_t)atoi(fields[6]);
27         gpsSatellites = (uint8_t)atoi(fields[7]);
28         if (quality == 0 ||
29             !GPS_StorePosition(fields[2], fields[3][0],
30                               fields[4], fields[5][0])) {
31             gpsFix = false;
32         }
33 // RMC chỉ hợp lệ khi trường trạng thái bằng A và vị trí qua kiểm tra.
34     } else if (isRMC && count > 6) {
35         if (fields[2][0] != 'A' ||
36             !GPS_StorePosition(fields[3], fields[4][0],
37                               fields[5], fields[6][0])) {
38             gpsFix = false;
39         }
40     }

```

41 }

Đoạn mã 6: Parser câu GGA/RMC sau khi checksum hợp lệ

7.7. Thu UART và phân loại trạng thái GPS

```
1 // Đọc hết byte chờ, cập nhật bộ đếm và thời điểm nhận gần nhất.
2 void GPS_Poll() {
3     while (gpsSerial.available() > 0) {
4         char c = (char)gpsSerial.read();
5         gpsByteCount++;
6         lastGPSByteMs = millis();
7
8         // Đô-la mở câu mới; xuống dòng kết thúc câu và gọi parser.
9         if (c == '$') {
10             gpsWorkingLength = 0;
11             gpsWorking[gpsWorkingLength++] = c;
12         } else if (c == '\n') {
13             if (gpsWorkingLength > 0) {
14                 gpsWorking[gpsWorkingLength] = '\0';
15                 GPS_ParseSentence(gpsWorking);
16                 gpsWorkingLength = 0;
17             }
18             // Bỏ CR và chống tràn bộ đệm 96 byte khi nối phần thân câu.
19             } else if (c != '\r' && gpsWorkingLength > 0 &&
20                 gpsWorkingLength < sizeof(gpsWorking) - 1) {
21                 gpsWorking[gpsWorkingLength++] = c;
22             }
23     }
24
25 // Phân biệt chưa nhận byte, mất UART, fix còn mới và đang nhận nhưng chưa fix.
26 const char *GPS_StatusText() {
27     if (gpsByteCount == 0) return "WAIT";
28     if (millis() - lastGPSByteMs > 3000) return "LOST";
29     if (gpsFix && millis() - lastGPSFixMs < 10000) return "FIX";
30     return "NOFIX";
31 }
```

Đoạn mã 7: Gom dòng NMEA và trả WAIT/LOST/NOFIX/FIX

8. Phân công thực hiện

Thành viên	Phần việc chính	Tỷ lệ
Phạm Hùng Tiến	Tích hợp, I2C/OLED, Wi-Fi AP, HTTP/DNS tự viết, khung dashboard, lịch chạy và kiểm thử tổng	33,4%
Hồ Sĩ Phú	MAX30102/PPG, BPM, SpO ₂ , chất lượng và hợp đồng dữ liệu PPG cho web	33,3%
Quách Gia Thịnh	IMU/GPS, té ngã, dữ liệu vị trí/chuyển động và thao tác điều khiển web	33,3%

Các phần việc được ghép và kiểm tra chung trên cùng một phiên bản chương trình.

9. Kiểm thử và kết quả

Phép thử	Kết quả đo/quan sát	Đánh giá
Quét I2C	Phát hiện 0x3C, 0x57, 0x68	Đạt
Định danh IMU	WHO_AM_I = 0x74	Đạt
Gia tốc nghỉ	1,07–1,08g	Đạt
Đếm bước khi đặt yên	Bộ đếm giữ nguyên 0	Đạt
Đếm bước mô phỏng	20 chu kỳ tổng hợp cho 20 bước; nhiều nhỏ/rung đơn cho 0	Đạt
Đếm bước khi đi bộ	Chưa có chuỗi đối chứng đủ dài để tính sai số	Cần thử thêm
MAX30102 rời tay	IR 330–400, kết quả ở WAIT	Đạt
MAX30102 đặt tay	IR 132.000–137.000	Đạt
Nhịp tim	BPM quan sát khoảng 72–83 qua các lần thử	Đạt
Thời gian PPG tổng hợp	BPM sơ bộ có thể dưới 1 giây, xác nhận theo RR; SpO ₂ từ khoảng 1 giây khi tín hiệu đạt chuẩn	Đạt
Dạng sóng PPG đảo	Bộ dò tự nhận cực tính dương/âm và vẫn khôi phục đúng BPM	Đạt
SpO ₂ nguyên mẫu	Khoảng 90–96% trong các lần thử gần nhất	Cần hiệu chuẩn

Phép thử	Kết quả đo/quan sát	Đánh giá
Luồng GPS	Hơn 7.000 byte NMEA; không mất UART	Đạt
GPS có vị trí	FIX, 4 vệ tinh ở lần chạy cuối, có vĩ/kinh độ	Đạt
Buzzer/LEDC	Kênh PWM khởi tạo thành công; chưa đo mức âm thực tế	Đã cài đặt
Wi-Fi Access Point	SSID health-monitor; IP 192.168.4.1; dashboard và API phản hồi	Đã cài đặt
Dashboard responsive	Hiển thị đủ PPG, bước, té ngã, thiết bị, GPS và trạng thái hệ thống	Đạt
Điều khiển web	Đặt lại bước và hủy cảnh báo trả JSON thành công	Đạt
Té ngã mô phỏng	Chuỗi đầy đủ vào ALERT; chuỗi thiếu va đập về NORMAL	Đạt
Té ngã thực tế	Chưa thử ngã người thật; chỉ mới kiểm tra logic và ngưỡng trên mô hình	Cần thử thêm
Biên dịch	969.218 byte flash (30%), RAM 49.972 byte (15%)	Đạt
Nạp phần cứng	Xác minh hash, hard reset thành công	Đạt

Các ca mô phỏng được tái lập bằng tệp `tests/algorithm_selfcheck.py`, chỉ dùng thư viện chuẩn Python và giữ nguyên hệ số/ngưỡng của firmware. Chúng chứng minh luồng quyết định chạy nhất quán nhưng không được dùng để thay thế số liệu đi bộ hoặc té ngã trên người thật.

Ảnh nguyên mẫu tại Hình 3 bổ sung bằng chứng trực quan cho phép thử tích hợp: ESP32-S3, màn hình, cảm biến quang, IMU, GPS và buzzer cùng hiện diện trên breadboard; OLED hiển thị dữ liệu trong khi người dùng đặt ngón tay lên MAX30102.

10. Đánh giá rủi ro và giới hạn

- SpO₂ chưa được hiệu chuẩn bằng thiết bị y tế tham chiếu, vì vậy chỉ mang tính học thuật.
- Chuyển động ngón tay, ánh sáng ngoài và lực ép làm giảm độ tin cậy PPG.
- GPS trong nhà có thể ở NOFIX hoặc số vệ tinh thấp; cần thử ngoài trời.

- Dashboard hiện dùng mạng cục bộ với một mật khẩu dùng chung; chưa có tài khoản, HTTPS hoặc phân quyền thao tác.
- Ngưỡng té ngã cần được đánh giá trên nhiều tư thế và đối tượng nhưng phải dùng đệm, dây giữ và quy trình an toàn.
- Nguyên mẫu chưa có quản lý pin, vỏ đeo, động cơ rung, nút hủy cảnh báo và kênh gửi cảnh báo từ xa.

11. Hướng phát triển

Nhóm đề xuất bổ sung nút SOS/hủy vật lý và động cơ rung; hiệu chuẩn SpO_2 ; ghi lịch sử đo; thêm xác thực web; dùng BLE/Wi-Fi STA/LTE để gửi cảnh báo ra ngoài mạng cục bộ; tối ưu tiêu thụ điện; thiết kế PCB và vỏ đeo. Đối với té ngã, ngưỡng cố định có thể được thay bằng mô hình thích nghi sau khi có bộ dữ liệu thử nghiệm đủ lớn.

12. Kết luận

Mô hình đã đọc cảm biến, hiển thị số đo trên OLED và web, nhận vị trí GPS, đồng thời điều khiển màn hình/loa khi máy trạng thái té ngã được kích hoạt. Kiến trúc không chặn cho phép cảm biến và dịch vụ HTTP/DNS tự viết dùng chung ESP32-S3. Trước khi sử dụng thực tế, sản phẩm cần được hiệu chuẩn, thử nghiệm an toàn và hoàn thiện bảo mật, nguồn, vỏ đeo cùng kênh truyền tin từ xa.

Tài liệu tham khảo

- Espressif Systems, ESP32-S3 Hardware Reference.
- Espressif Systems, Arduino Core for ESP32 và mã nguồn `Wire` đi kèm.
- Analog Devices, MAX30102 Datasheet, Revision 1.
- Analog Devices, hướng dẫn hiệu chuẩn phép đo SpO_2 .
- Freenove, ESP32-S3-WROOM Camera Board, dòng FNK0085; nguồn ảnh bo điều khiển.
- Waveshare, OLED SSD1306 0,96 inch 128x64 và mô-đun GPS UART NEO-6M; nguồn ảnh tham chiếu.
- DFRobot, Fermion MAX30102 V1.0; nguồn ảnh bo breakout tham chiếu.
- TDK InvenSense, MPU-6000/MPU-6050 Register Map, Revision 4.2; dùng để đối chiếu nhóm thành ghi gia tốc tương thích.
- Solomon Systech, danh mục bộ điều khiển OLED SSD1306 128x64.